CampusWay

Tags	
Created by	Automation

1. Proposed database schema v1

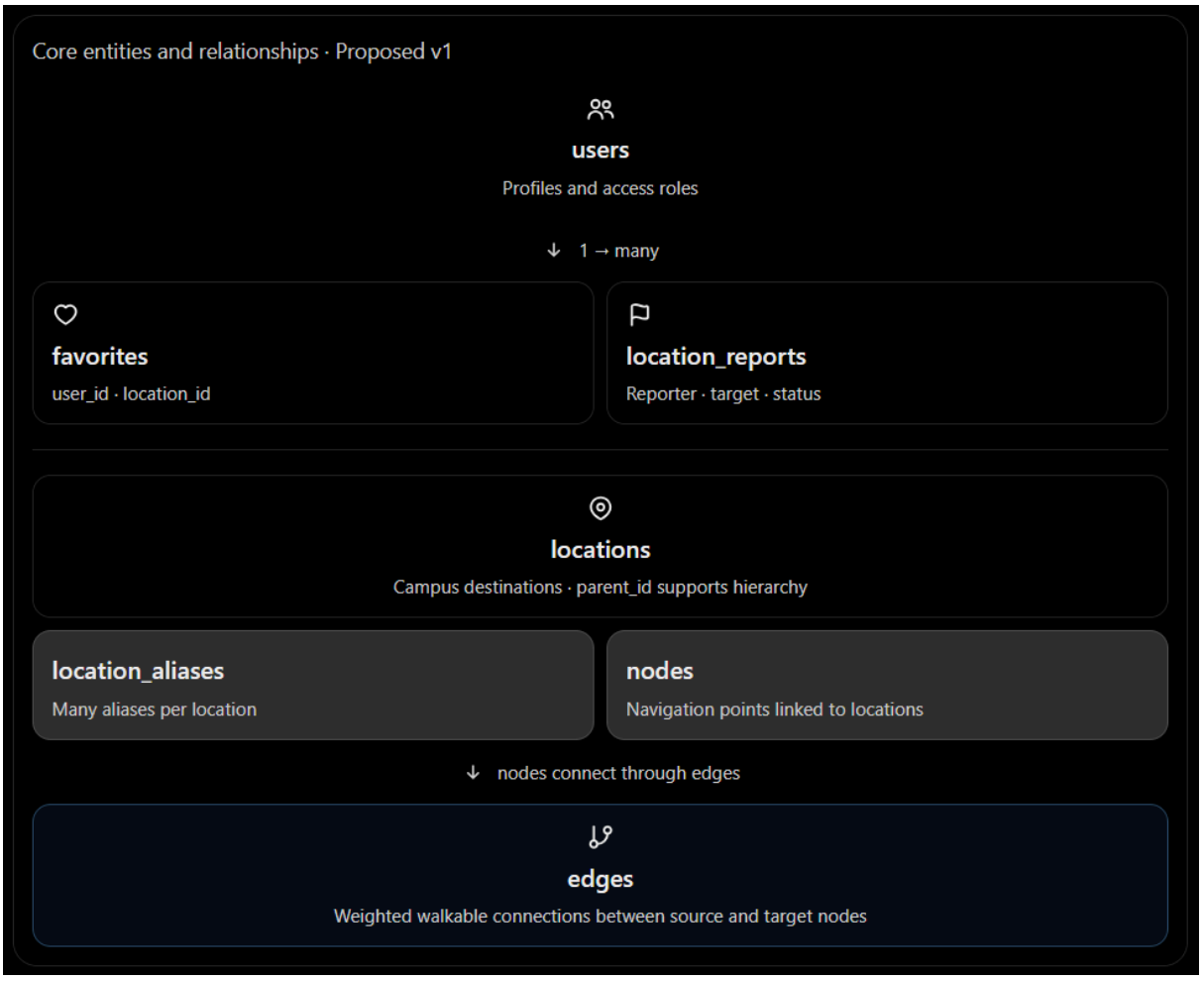
Table	Purpose
`users`	User identity, profile and role (pending authentication implementation).
`locations`	All campus destinations: blocks, buildings, food courts, gates, departments and rooms.
`nodes`	Junctions, entrances and other points in the navigation graph.
`edges`	Walkable connections between nodes, with distance and routing attributes.
`location_aliases`	Alternative names and abbreviations for campus search.
`favorites`	Locations saved by users.
`location_reports`	Reports of missing locations, incorrect pins or blocked paths.

Technical decisions

- Database: Shared Neon PostgreSQL.
- Primary keys: UUIDs for the main entities.
- Relationships: Foreign keys with appropriate deletion rules.
- Location hierarchy: Self-referencing `parent_id` in `locations`.
- Routing: Separate graph nodes and edges, supporting weighted shortest-path algorithms.
- Coordinates: Latitude and longitude initially; consider PostGIS when advanced spatial queries become necessary.
- Schema management: Version-controlled migrations committed and pushed to Git.
- Authentication: Harsh owns the authentication integration; the user model must be agreed upon before implementation.

- Data access: Backend services validate requests and enforce permissions before database changes.

Entity relationship diagram



2. Detailed table definitions

These are the proposed columns, data types, constraints and indexing decisions. UUIDs are the primary-key standard, and timestamps use PostgreSQL's `TIMESTAMPTZ`.

A. `users` — user profiles

Column	Type	Constraint / purpose
`id`	UUID	Primary key
`full_name`	VARCHAR(120)	Nullable until profile completion

Column	Type	Constraint / purpose
`email`	VARCHAR(255)	Nullable; unique when present
`phone`	VARCHAR(20)	Nullable; unique when present
`role`	VARCHAR(20)	Default `student`; constrained allowed values
`is_active`	BOOLEAN	Default `TRUE`
`created_at`	TIMESTAMPTZ	Default current timestamp
`updated_at`	TIMESTAMPTZ	Updated on profile changes

Important: The `users` table represents the application profile, not the authentication mechanism itself. Can connect it to the authentication provider he/she selects.

B. `user_identities` — authentication linkage

This additional table is recommended because the application intends to support both Google sign-in and mobile OTP.

Column	Type	Constraint / purpose
`id`	UUID	Primary key
`user_id`	UUID	FK → `users.id`
`provider`	VARCHAR(30)	E.g. `google` or another supported provider
`provider_subject`	VARCHAR(255)	Provider's stable unique user identifier
`created_at`	TIMESTAMPTZ	Default current timestamp

Constraints: unique `(provider, provider_subject)` and an index on `user_id`.

Do not store plaintext OTPs, authentication tokens or passwords in this table. The identity provider or authentication service should manage the relevant credentials.

C. `locations` — campus places

Column	Type	Constraint / purpose
`id`	UUID	Primary key
`parent_id`	UUID	Nullable self-FK → `locations.id`
`name`	VARCHAR(180)	Required
`slug`	VARCHAR(200)	Unique, URL-friendly identifier
`location_type`	VARCHAR(40)	Building, block, room, gate, food court, etc.

Column	Type	Constraint / purpose
`description`	TEXT	Nullable
`latitude`	DOUBLE PRECISION	Nullable until coordinates are verified
`longitude`	DOUBLE PRECISION	Nullable until coordinates are verified
`address_details`	TEXT	Optional additional location details
`is_verified`	BOOLEAN	Default `FALSE`
`is_active`	BOOLEAN	Default `TRUE`
`created_at`	TIMESTAMPTZ	Creation timestamp
`updated_at`	TIMESTAMPTZ	Modification timestamp

Design decision: A block can contain buildings; a building can contain floors or rooms. The hierarchy is flexible, but the application must validate allowed parent-child combinations and prevent circular relationships.

Coordinates must satisfy latitude −90 to 90 and longitude −180 to 180 when supplied.

D. `location_aliases` — search synonyms

Column	Type	Constraint / purpose
`id`	UUID	Primary key
`location_id`	UUID	FK → `locations.id`
`alias`	VARCHAR(180)	Required alternative name
`normalized_alias`	VARCHAR(180)	Normalized search representation
`created_at`	TIMESTAMPTZ	Creation timestamp

Add an index on `normalized_alias` and consider uniqueness on `(location_id, normalized_alias)`.

E. `nodes` — routing graph vertices

Column	Type	Constraint / purpose
`id`	UUID	Primary key
`location_id`	UUID	Nullable FK → `locations.id`
`node_type`	VARCHAR(40)	Junction, entrance, crossing, staircase, etc.
`latitude`	DOUBLE PRECISION	Required node coordinate
`longitude`	DOUBLE PRECISION	Required node coordinate

Column	Type	Constraint / purpose
`is_accessible`	BOOLEAN	Accessibility classification; default `TRUE`
`is_active`	BOOLEAN	Default `TRUE`
`created_at`	TIMESTAMPTZ	Creation timestamp

F. `edges` — walkable graph connections

Column	Type	Constraint / purpose
`id`	UUID	Primary key
`source_node_id`	UUID	FK → `nodes.id`
`target_node_id`	UUID	FK → `nodes.id`
`distance_m`	DOUBLE PRECISION	Required, greater than zero
`path_type`	VARCHAR(40)	Footpath, road crossing, stairs, ramp, etc.
`is_bidirectional`	BOOLEAN	Default `TRUE`
`is_accessible`	BOOLEAN	Whether the connection meets defined accessibility criteria
`is_active`	BOOLEAN	Default `TRUE`
`created_at`	TIMESTAMPTZ	Creation timestamp
`updated_at`	TIMESTAMPTZ	Modification timestamp

Constraints: source and target must differ; distance must be positive; duplicate connections should be prevented according to the chosen directional-edge policy.

For the routing engine, use `distance_m` as the initial edge weight. Later, additional routing costs can account for stairs, slopes, accessibility or estimated travel time.

G. `favorites` — saved places

Column	Type	Constraint / purpose
`user_id`	UUID	FK → `users.id`
`location_id`	UUID	FK → `locations.id`
`created_at`	TIMESTAMPTZ	Save timestamp

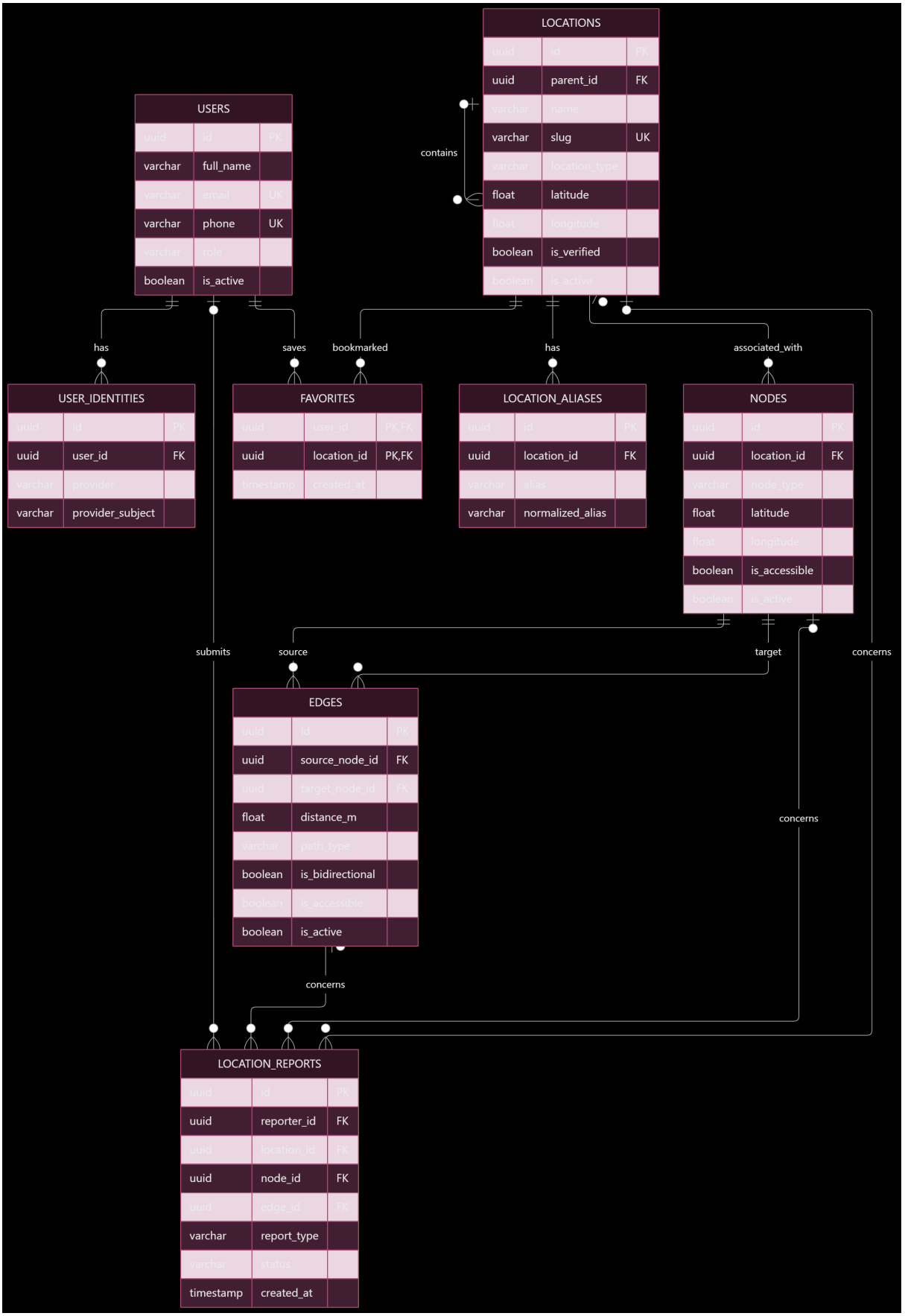
Use the composite primary key `(user_id, location_id)` so a user cannot save the same location twice.

H. `location_reports` — map correction requests

Column	Type	Constraint / purpose
`id`	UUID	Primary key
`reporter_id`	UUID	Nullable FK → `users.id`
`location_id`	UUID	Nullable FK → `locations.id`
`node_id`	UUID	Nullable FK → `nodes.id`
`edge_id`	UUID	Nullable FK → `edges.id`
`report_type`	VARCHAR(40)	Missing place, wrong pin, blocked path, etc.
`description`	TEXT	Report details
`status`	VARCHAR(20)	Default `pending`
`created_at`	TIMESTAMPTZ	Submission timestamp
`resolved_at`	TIMESTAMPTZ	Nullable resolution timestamp

At least one target reference should be provided, and the application should validate that the report targets a valid entity. A later iteration can add reviewer IDs and an audit trail.

3. ER diagram in Mermaid


LOCATIONS
uuid id PK
uuid parent_id FK
varchar name
varchar slug UK
varchar location_type
float latitude
float longitude
boolean is_verified
boolean is_active
USERS
uuid id PK
varchar full_name
varchar email UK
varchar phone UK
varchar role
boolean is_active
contains
has
saves
bookmarked
has
associated_with
USER_IDENTITIES
uuid id PK
uuid user_id FK
varchar provider
varchar provider_subject
FAVORITES
uuid user_id PK,FK
uuid location_id PK,FK
timestamp created_at
LOCATION_ALIASES
uuid id PK
uuid location_id FK
varchar alias
varchar normalized_alias
NODES
uuid id PK
uuid location_id FK
varchar node_type
float latitude
float longitude
boolean is_accessible
boolean is_active
submits
source
target
concerns
EDGES
uuid id PK
uuid source_node_id FK
uuid target_node_id FK
float distance_m
varchar path_type
boolean is_bidirectional
boolean is_accessible
boolean is_active
concerns
concerns
LOCATION_REPORTS
uuid id PK
uuid reporter_id FK
uuid location_id FK
uuid node_id FK
uuid edge_id FK
varchar report_type
varchar status
timestamp created_at